软件质量保证与测试

——编码过程中的测试



黑盒测试落地的测试工具

Java

JUnit：最广泛使用的开源框架，支持注解驱动测试和参数化测试，最新版本为 **JUnit 5**。

TestNG：支持多线程测试、分组测试和依赖管理，适合复杂场景。

JUnit

JUnit是一个开放源代码的**Java**测试框架，用于编写和运行可重复的测试。**JUnit**测试是程序员测试（白盒测试/黑盒测试），因为程序员知道被测试的软件如何（**How**）完成功能和完成什么样（**What**）的功能。

JUnit 5.x (2017-present): 继续发展并引入了模块化设计，提供了一个更现代化的测试框架。

链接地址：

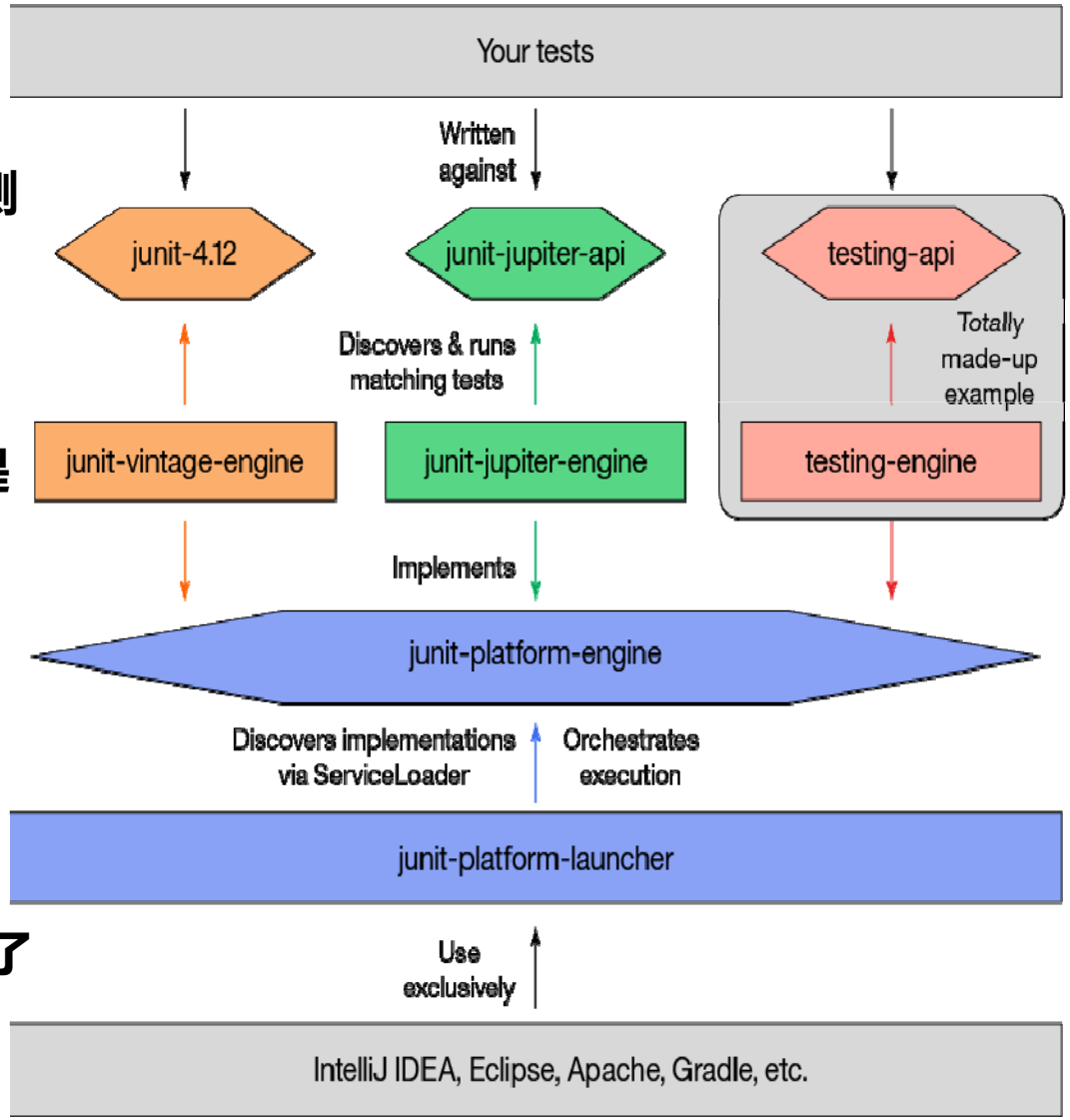
https://blog.csdn.net/weixin_56738054/article/details/130139224

Csdn:搜索 “JUnit5单元测试框架的使用教程与简单实例”

JUnit 5架构

JUnit 5 = JUnit Platform + JUnit Jupiter + JUnit Vintage（复古）

- ✓ **JUnit Platform**：基于JVM的执行测试的基础框架及其Test-Engine API，还提供了一个控制台启动器（以命令行启动平台），为Gradle和Maven构建插件支持CI/CD，同时提供基于JUnit 4的Runner。
- ✓ **JUnit Jupiter**：在JUnit 5中编写测试和扩展的新编程模型和扩展模型的组合。Jupiter子项目提供了Test-Engine在平台上运行基于Jupiter的测试。
- ✓ JUnit Vintage（兼容模型）：提供了TestEngine在平台上运行基于JUnit 3和JUnit 4的测试



Junit框架常用注解

@Test注解

表示这个方法是一个测试用例。当@Test注解作用在一个方法上面的时候，表示这一个方法是可以直接运行的，无需通过main方法来调用。

expected 用于异常测试，检查测试方法是否抛出指定异常，如：@Test(expected = ArithmeticException.class)

timeout 用于限时测试，检查方法的执行所耗费的毫秒数是否在指定时间内，如：@Test(timeout = 1000)

需要了解JUnit的关键组件

Assert类

Assert（断言）类是**JUnit**框架中非常重要的类，用于判断所返回的实际结果与预期结果是否相符。

Annotation（注释）

JUnit中以@引导标注测试类中不同方法的功能。

Runner（运行器）

JUnit通过运行器**Runner**和显示测试执行结果。



Junit框架常用注解

@BeforeEach

在执行每一个被@Test修饰的方法之前，都会执行一次@BeforeEach注释的方法。

@BeforeAll

首先，被@BeforeAll注释的方法一定是静态的。

其次，不同于@BeforeEach的是，@BeforeAll注释的方法一定是静态的，并且只会在加载外部类的时候执行一次，并不会在每一个方法执行之前再次执行了。

@AfterAll

被这个注解修饰的方法也必须是静态的，在每一个@Test之后进行修饰。

@AfterEach

在每一个@Test注释的方法执行结束的末尾，都会执行一次@AfterEach注释的方法。

@Disabled 用于禁用测试类或者测试方法。

测试套件(Suit)

测试套件，用于把所有的测试用例都执行起来。

@Suite

这一个注解作用于类上面，用于执行所有的测试用例。

@SelectClasses({待测试的类.class})

这个注解和上面的注解需要配合使用，这样才可以一起执行待测试类的测试用例。

当然也可以传递多个**.class**对象。

```
1
2  @Suite
3  @SelectClasses({Test1.class,Test2.class})
4  public class SuitTest {
5
6  }
```

测试套件(Suit)

测试套件，用于把所有的测试用例都执行起来。

@Suite

这一个注解作用于类上面，用于执行所有的测试用例。

@SelectPackages("包名")

扫描这一个包下面包含了@Test注解的方法的类。

```
1  @Suite
2  @SelectPackages("TestFile")
3  public class SuitTest {
4
5  }
```

参数化

@ParameterizedTest、@ValueSource(strings = {})

第一个注解**@Parameterized**作用在方法上面，用来表示这个方法支持传入多个参数；

第二个注解**@ValueSource**用于指定传入参数的类型；

例如：

strings={}就表示传入的是一个**string**类型的数组；

ints={2，3，4，5}表示传入的参数是一个**int**类型的数组；

但是，同一个**@ValueSource**注解当中只支持一种数据类型。

```
@ParameterizedTest
@ValueSource(ints = {2,3,4,5})
public void parametersTest(int age){
    System.out.println(age);
}
```

参数化

@ParameterizedTest、@CsvSource(value={"第一组数据","第二组数据"})

第一个注解**@Parameterized**作用在方法上面，用来表示这个方法支持传入多个参数；

第二个注解**@CsvSource**每一组数据当中对应方法传入的参数的值。采用","来分割；

```
1  @ParameterizedTest
2      @CsvSource(value = {"小明,20","小红,30","小李,40"})
3      public void printStringAndInteger(String name,int age){
4          System.out.println("name:"+name+";age="+age);
5      }
```

参数化

@ParameterizedTest、@CsvFileSource(resources="/文件名称")

读取文件参数

第一步：在**resources**目录下面新建一个文件，并且指定名称；

第二步：在文件当中以逗号的方式分割每一组参数；

```
@ParameterizedTest
@CsvFileSource(resources = "/my.csv")
public void printStringAndAge3(String name,int age){
    System.out.println("name:"+name+";age="+age);
}
```

读取其他磁盘文件的操作：指定文件为files=...

执行顺序

测试类执行顺序：

@BeforeAll –> @BeforeEach –> @Test –>

@AfterEach –> @AfterAll

测试方法执行顺序：

@ BeforeEach –> @Test –> @ AfterEach

如果有继承则各注解需要先执行父类的方法

断言 - 执行结果的验证

写自动化测试，结果要么是成功的，要么是失败的，不存在成功一半的情况。那么就需要使用到断言:assert，断言预期结果和真实结果是否一致。

fail 标注测试方法失败

assertTrue 判断是否为**True** **assertFalse** 判断是否为**False**

assertSame 两个对象是否相等，比地址

assertNotSame两个对象是否不相等

assertNull 对象是否为**Null** **assertNotNull** 对象是否为非**Null**

assertArrayEquals 比较两个数组是否相等

assertEquals 比较两个对象是否相等，比值 **assertNotEquals**

assertThat 验证是否匹配**Matcher**定义的接口

假设 - 测试条件验证

假设机制用来判断测试之前是否满足了某种条件，假设不成功不代表测试失败，而是测试所需的条件没有满足。**JUnit的默认Runner**会忽略掉这个假设条件所在的测试方法。

```
//only provides information if database is reachable.
@Test
public void calculateTotalSalary() {
    DBConnection dbc = Database.connect();
    assumeNotNull(dbc);
    // ...
 }
```

假设语句

assumeNoException(Throwable t)

无异常

assumeNotNull(Object... objects)

对象非空

assumeTrue(boolean b)

类型为True

assumeThat(T actual, Matcher<T> matcher)

验证是否匹配Matcher定义的接口

如果在@BeforeClass标注的方法中使用了Assume，假设条件不满足的话，那么除了@AfterClass所标注的方法外其他都不会执行。@Before和@After注解也类似。

测试套件

- 将多个测试用例组合在一起形成一个套件。这样只要执行测试套件，即可运行全部测试，而不需要分别运行。
- 支持@BeforeClass和@AfterClass注解，所以套件的执行顺序为先执行套件的@BeforeClass注解的方法，然后按照原来的顺序执行，最后执行套件@AfterClass注解的方法

```
import org.junit.runner.RunWith;
import org.junit.runners.Suite;

@RunWith(Suite.class)
@Suite.SuiteClasses({JunitTest1.class,JunitTest2.class})
public class JunitTest5 {
        ……
}
```

TestNG（ https://blog.csdn.net/m0_53753920/article/details/125018749 ）

TestNG(Test Next Generation, 下一代测试技术）在**JUnit**和**Nunit**基础上新增了许多功能。**TestNG**支持单元自动化测试，另外，它可以集成**selenium， Appium**和**HttpClient**等框架做各种类型的自动化测试。

TestNG测试框架， 简化从单元测试到集成测试的广泛测试需求。

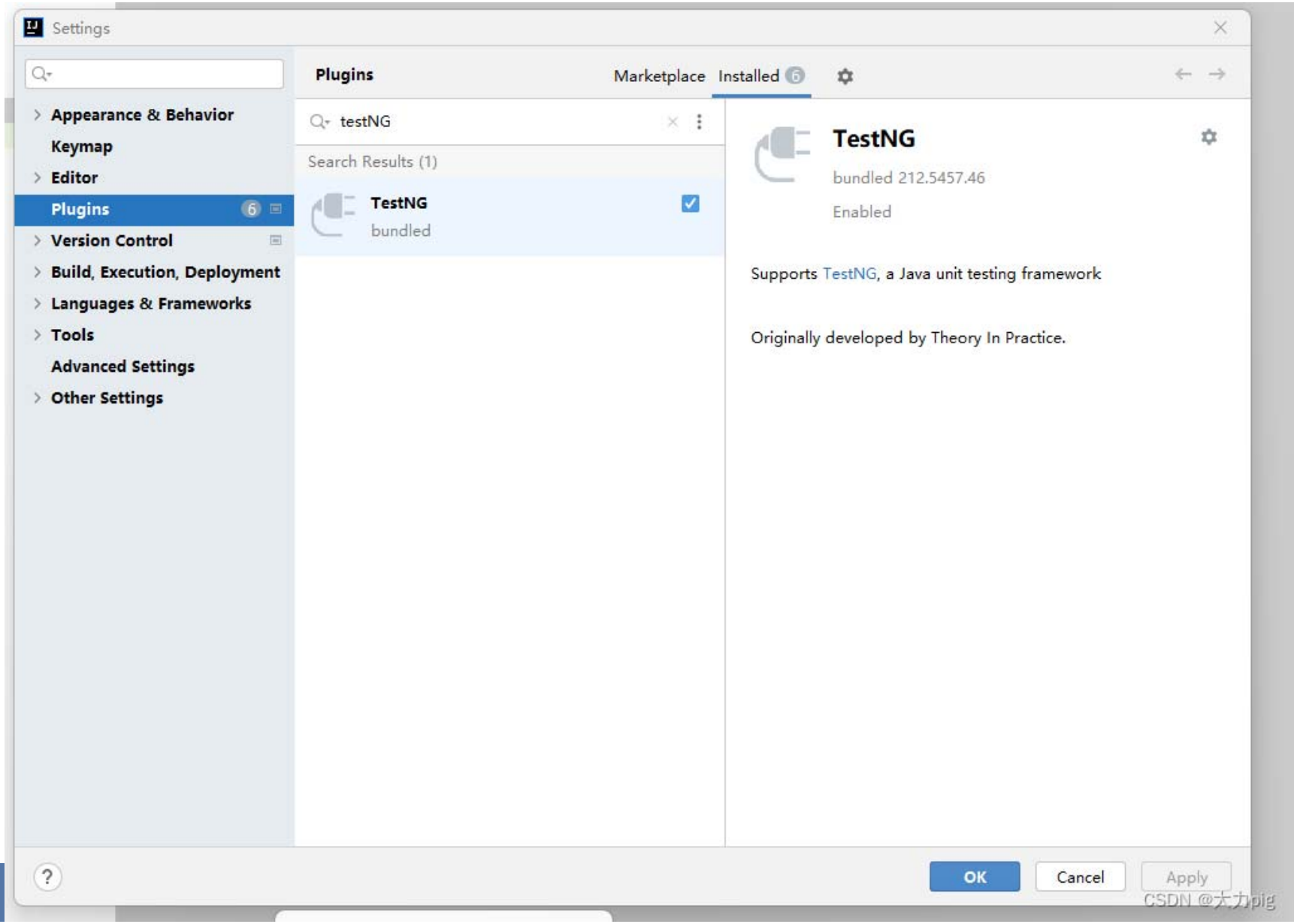
提供了一系列丰富的注解（标签），方便的编写和管理测试用例和测试方法。

1.强大的注解功能。 **2.** 数据驱动。 **3.** 灵活的测试配置。

4. 支持多种并发测试策略。

5. 可以和多种工具（插件）协同使用，如**eclipse, IDEA, Maven**等。

IDEA安装TestNG插件



CSDN @大力pig

test NG依赖

```
<dependency>
    <groupId>org.testng</groupId>
    <artifactId>testng</artifactId>
    <version>6.11</version>
</dependency>
```

TestNG注解

（1）前置条件和后置条件 把注解作为前置条件（或初始化操作）和后置条件（或清理操作）使用。

（2）数据驱动 即测试用例和测试数据分离，以便于维护和管理。

（3）测试用例 该分类只有一个@Test注解，@Test注解的作用是对测试用例进行控制。

（4）监听器 该分类只有一个@Listeners注解。监听器的作用是控制测试过程，如果采用默认监听器， 则不需要任何配置。如果使用自定义监听器，则需要使用@Listeners注解或者TestNG.xml进行配置。

TestNG注解

Test注解使用此方法注解的类将被**TestNG所识别**并运行。

属性：

dependsOnMethods: 依赖的方法

alwaysRun: 设置为**True**时， 无论什么时候都会运行。

description: 代表测试用例描述。

enabled: 设置为**false**时失效

dataProvider: 数据提供者的名称

dependsOnMethods

dependsOnMethods代表依赖一个或者多个方法，**dependsOnGroups**代表依赖一个或者多个分组，一旦被依赖的测试用例执行失败，则TestNG将跳过该测试用例。建议：尽量不要使用dependsOnMethods或者dependsOnGroups标签，这违背了测试用例需要解耦的原则。

```
@Test
  public void test1(){
     System.out.println("test1.....");
  }

  @Test(dependsOnMethods = "test1")
  public void test2(){
     System.out.println("test2.....");
  }
```

```
"C:\Program Files\Java\jdk1.8.0_202\b
test1.....
test2.....

===============================================
Default Suite
Total tests run: 2, Failures: 0, Skip
===============================================
```

TestNG.xml 文件的使用

TestNG.xml文件的作用是控制测试执行的过程，该文件可以使用Eclipse的TestNG插件自动生成。

在工程右键-> TestNG-> Convert to TestNG选项，此时会弹出” Refactoring" 对话框，直接单击“Finish" 按钮， 生成testng.xml文件。

suite: 套件， 项目， 最大。test: 模块。classes, class: 测试类。

```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <!DOCTYPE suite SYSTEM "http://testng.org/testng-1.0.dtd">
3  <suite name ="Suite" parallel="false">
4      <test name = "test">
5          <packages>
6              <package name="com.testng"></package>
7          </packages>
8      </test>
9  </suite>
```

TestNG.xml 文件的使用-package标签使用

一个class需要一个<class>标签，那么10个class则需要10个<class>标签。class在一个package中，则可以使用<package>进行package设置，以简化配置过程。

运行方法也是一样，Run as -> testNG

```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <!DOCTYPE suite SYSTEM "http://testng.org/testng-1.0.dtd">
3  <suite name ="Suite" parallel="false">
4      <test name = "test">
5          <packages>
6              <package name="com.testng"></package>
7          </packages>
8      </test>
9  </suite>
```

include和exclude标签代码

<include>和<exclude>标签的作用是对方法进行设置，<include>代表包含， <exclude>代表排除。

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM "http://testng.org/testng-1.0.dtd">
<suite name ="Suite" parallel="false">
    <test name = "test">
        <classes>
            <class name="com.testng.Demo">
                <methods>
                    <include name="test."/>
                    <exclude name="test3"/>
                </methods>
            </class>
        </classes>
    </test>
</suite>
```

〈parameter〉

<parameter>标签和@Parameters注解配合使用，可以对测试用例传递参数，达到数据分离的 效果。

```
1  public class Demo8 {
2      @Parameters({"myParam"})
3      @Test
4      public void testCase1(String param) {
5          System.out.println("myParam is : " + param);
6      }
7  }
```

<parameter>

testng xml文件如下：

```xml
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM "http://testng.org/testng-1.0.dtd">
<suite name="Suite" parallel="false">
    <parameter name="myParam" value="myParamValue"/>
    <test name="test">
        <classes>
            <class name="com.testng.Demo8"/>
        </classes>
    </test>
</suite>
```

```
"C:\Program Files\Java\jdk1.8.0_202\bin\ja
myParam is : myParamValue

===============================================
Suite
Total tests run: 1, Failures: 0, Skips: 0
===============================================
```

myParam的值通过<paramet
解配合传递给了testCase1.

<parameter>

testng xml文件如下：

```xml
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM "http://testng.org/testng-1.0.dtd">
<suite name="Suite" parallel="false">
    <parameter name="myParam" value="myParamValue"/>
    <test name="test">
        <classes>
            <class name="com.testng.Demo8"/>
        </classes>
    </test>
</suite>
```

```
"C:\Program Files\Java\jdk1.8.0_202\bin\ja
myParam is : myParamValue

===============================================
Suite
Total tests run: 1, Failures: 0, Skips: 0
===============================================
```

myParam的值通过<paramet
解配合传递给了testCase1.

TestNG测试报告

运行testNG后， 工程中会多一个test-output目录。 展开test-output目录， 在emailable-repprt.html上用鼠标右击，从弹出的快捷键菜单中选择“ Open with -> Web Browser”选项，

Html格式的测试报告

TestNG还支持与第三方测试报告的集成， 常见的包括ReportNG，Extent Reporting，和Allure等。

testNG也支持自定义报告